ConstructsHub: Full Implementation Plan with AI Agents

Tools: Antigravity (planner + inspector) · Claude Code (senior foreman) · OpenCode + Nemotron Ultra 3 (bulk crew)

Inputs used: Master SRS v2.0 (12 bounded contexts, 3 AI services), the 17 screens PDF, the October 2026 Project Status Report, and the integration playbook.

Where the project stands today: Core Platform 81%, Search & AI 73%, Payments 73%, Marketplace 62%, Clients/Admin 52%. The app looks finished but has fake-success spots (adapters that return true, dispatch that picks the first driver, sealed bids that are not encrypted) and 8 launch-gate blockers. This plan closes those first, then finishes the marketplace, then the AI modules, then polish.

How to use this plan

Three agents, three jobs, one repo. Every feature below has three prompts. The rules:

> Work **one feature at a time**. Never give any agent two features at once.
>
> **Antigravity plans and verifies.** Its prompt comes first: it turns the SRS section into tickets and, at the end, opens the running app in its browser to prove the feature works.
>
> **Claude Code owns the hard core:** money, security, crypto, AI logic, anything with real risk.
>
> **OpenCode + Nemotron Ultra 3 owns the bulk:** screens, CRUD, forms, tests, i18n, docs. If it fails the same ticket twice, that ticket moves to Claude Code.
>
> **Every prompt starts with the Context Header** (next section). Paste it above the feature prompt, always.
>
> Every ticket ends with proof: test output, or an Antigravity browser screenshot. No proof, not done. Commit after every green feature.

The Context Header (paste this at the top of EVERY prompt)

Code

```
PROJECT: ConstructsHub (BanahBuild). Construction marketplace for Saudi Arabia.
STACK: Modular monolith. Core API: NestJS + PostgreSQL (+PostGIS, +pgvector),
OpenSearch,
```

```
Redis, S3-compatible storage, BullMQ workers, WebSocket gateway. Web: Next.js App
Router +
TypeScript + Tailwind. Mobile: React Native + Expo (Expo Router). AI services:
Python
FastAPI (nlp-service, vision-service, bim-service with IfcOpenShell). Contracts
live in
packages/contracts (Prisma schema + OpenAPI 3.1 + generated typed client). One
source of
truth: change the contract once, regenerate, never hand-edit generated files.
RULES: Read AGENTS.md / CLAUDE.md before writing anything. One feature per
session.
Write or update a test for every behavior you add. Never leave a stub that
returns true,
a TODO, or a mocked result presented as real. Run type-check + lint + relevant
tests and
show me the output before saying done. Money, auth, crypto and permissions code
always
needs a test. Commit only after everything is green.
DEFINITION OF DONE: code + tests + passing CI + (UI features) a screenshot of the
running
screen. No exceptions.
```

Execution levels at a glance

Level	Name	Features	Lead agent	Why this order
0	Foundation lock	F0.1, F0.2	Antigravity + Claude	Contracts, CI and the stub list make everything after it honest
1	Launch-gate closers	F1.1 – F1.8	Claude Code	The 8 blockers. Nothing ships without these
2	Marketplace completion	F2.1 – F2.8	OpenCode bulk + Claude on money logic	The revenue-bearing core, currently the weakest band
3	AI modules	F3.1 – F3.5	Claude Code	The differentiators, built on hardened services
4	Polish & scale	F4.1 – F4.6	OpenCode bulk + Antigravity audit	i18n, a11y, analytics, performance

Estimated mix: roughly 25% Claude Code, 60% OpenCode (Nemotron Ultra 3), 15% Antigravity. That protects your Claude credits while keeping risk on the strong model.

LEVEL 0 — FOUNDATION LOCK

F0.1 Contracts, rules files and CI gates

Status: partially exists. **Lead:** Claude Code (setup) · **Antigravity:** audit · **OpenCode:** fills gaps

Antigravity prompt:

Code

```
[Context Header]
You are the planner. Read the whole repo structure and Master SRS v2.0 section 3
(Architecture) and section 4 (Bounded Contexts). Produce: (1) a one-page map of
the
monorepo showing where contracts, services, web, mobile and AI services live;
(2) a checklist of every contract file (Prisma, OpenAPI, generated clients) with
the
commands that regenerate them; (3) a backlog of the 6 CI jobs this repo needs
(type-check, lint, unit tests, contract-drift check, build, E2E smoke) with the
exact
command each should run; (4) the order to build them in. Output as a markdown
file
docs/plan/level0-foundation.md. Do not write application code.
```

Claude Code prompt:

Code

```
[Context Header]
Set up the foundation gates so no agent can ship broken code again:
1. Create/repair CLAUDE.md and AGENTS.md with identical rules: stack summary,
folder
   map, contract regeneration commands, "no stubs", "no done without test
output",
   commit discipline. Keep both under 150 lines.
2. Write the GitHub Actions workflow .github/workflows/ci.yml with jobs: type-
check,
   lint, unit tests, contract-drift (regenerate contracts and fail if git diff is
   non-empty), docker build for core API. All must pass on PR.
3. Add a pre-commit hook running type-check on changed packages.
4. Add Makefile targets: make contracts, make test, make up, make e2e.
Run the full pipeline once and show me green output. Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Fill the gaps Antigravity's audit (docs/plan/level0-foundation.md) found in the
rules files and CI: add missing Makefile targets, add the contract-drift check
script (packages/contracts/scripts/check-drift.sh), and write docs/ONBOARDING.md
explaining in 20 lines how a new contributor (human or AI agent) gets the repo
running. Run type-check and show output. Commit.
```

F0.2 Stub hunt and kill list

Lead: Antigravity (find) → Claude Code (fix risky ones) → OpenCode (fix safe ones)

Antigravity prompt:

Code

```
[Context Header]
Hunt every fake success in this repo. The status report already found: social
media
adapters (Instagram/TikTok/Facebook) that return true without calling any API,
```

```
dispatch
that uses findFirst instead of a real algorithm, sealed bids "hidden" only by
query
filter, mobile location endpoints that 404. Search the ENTIRE repo (core API,
workers,
web, mobile, AI services) for: hardcoded `return true` in service methods,
TODO/FIXME/
placeholder/mock markers, endpoints referenced by clients but not implemented,
and
integrations that log instead of call. Output docs/plan/stub-kill-list.md as a
table:
file, line, what it fakes, risk (money/security/core/UX), suggested owner (claude
or
opencode). Do not fix anything yourself.
```

Claude Code prompt (for each risky stub, e.g. sealed bids):

Code

```
[Context Header]
Ticket from docs/plan/stub-kill-list.md, row: [paste row].
Implement the real behavior. For sealed bids: bids must be encrypted at rest with
an
envelope key (AES-256-GCM, key from env/KMS), unreadable until tender close, with
an
audit log of every decryption. Replace query-filter hiding with real crypto.
Write
unit + integration tests proving: a normal admin query cannot read a sealed bid,
and
after close the reveal works. Show test output. Commit.
```

OpenCode + Nemotron Ultra 3 prompt (for each safe stub):

Code

```
[Context Header]
Ticket from docs/plan/stubkill-list.md, row: [paste row]. Risk marked UX/core.
Implement the real behavior described in the referenced SRS section. If the fix
needs
a new dependency or touches money/auth/crypto, STOP and report instead of doing
it.
Write a test, run type-check + tests, show output. Commit.
```

LEVEL 1 — LAUNCH-GATE CLOSERS

The status report's 8 blockers, in build order. Level 1 is done when all 8 are closed.

F1.1 Saudi payment gateway adapter (real, not simulated)

Lead: Claude Code. **Money logic: never OpenCode.**

Antigravity prompt:

Code

```
[Context Header]
Read SRS sections on Payments & Ledger context and the status report's payments
table
(gateway/wallet/ledger/escrow at ~70%). Produce a ticket list for finishing the
payment
stack: gateway adapter contract, wallet top-up, ledger double-entry integrity,
escrow
state machine (held, released, refunded, disputed). For each ticket give
acceptance
tests. Flag which tickets involve money math (must go to Claude Code). Output
docs/plan/level1-payments.md.
```

Claude Code prompt:

Code

```
[Context Header]
Implement the real Saudi payment gateway adapter behind the existing
PaymentGateway
interface (pick the provider per your business account: Moyasar or HyperPay).
Requirements: create/verify/capture/refund flows, webhook endpoint with signature
verification and idempotency keys, saved-token handling for repeat payments, 3DS
redirect flow for web and mobile, full request/response logging with card data
redacted (never store PAN). Wire it into escrow: payment success moves funds to
escrow ledger entries, refund reverses them. Include a SandboxMode that calls the
provider's real sandbox, not a fake. Write integration tests against the sandbox
(vitest, mocked HTTP at the boundary only). Show test output. Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Support work for the payment adapter (do NOT touch the adapter itself): (1) write
web/checkout and mobile/checkout UI states for the 3DS redirect: pending,
awaiting
bank, success, failure, retry; (2) add all gateway error codes to a shared
payments-errors.ts with Arabic + English messages; (3) write unit tests for the
wallet top-up DTOs and validation. Show test output. Commit.
```

F1.2 ZATCA Fatoora e-invoicing (sandbox + CSID)

Lead: Claude Code.

Claude Code prompt:

Code

```
[Context Header]
Implement ZATCA Phase-2 e-invoicing integration in the Billing context, using the
official Fatoora sandbox: (1) CSID onboarding flow (renew/produce); (2) build
invoices as UBL 2.1 XML with the KSA extensions, QR code (TLV base64) per ZATCA
spec;
(3) hash (SHA256), sign with org certificate, submit to the sandbox reporting
API,
store clearance/reporting status + warnings per invoice; (4) a BullMQ retry
```

```
worker
with backoff for transient failures and a dead-letter path; (5) an invoice state
machine: draft, signed, reported/cleared, accepted, rejected with reason. Add a
compliance test suite that validates one standard invoice and one simplified
invoice
against the sandbox. Show test output and the generated QR payload for one sample
invoice. Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build the Finance UI screens that show ZATCA status (do not touch the
integration):
web/finance/invoices list with status badges
(draft/signed/reported/cleared/rejected),
a detail drawer showing the QR, submission warnings and retry button, and a
filter
bar. Use the generated typed client. RTL-safe (test both directions). Add
Playwright
component tests for the badges and filters. Show test output. Commit.
```

F1.3 Escrow release cron + milestone ledger completion

Lead: Claude Code.

Claude Code prompt:

Code

```
[Context Header]
Finish escrow and the milestone ledger to 100%: (1) implement the escrow release
cron
(BullMQ repeatable job): on milestone acceptance + dispute window elapsed,
release
funds to the contractor's ledger with a double-entry journal row and an audit
event;
auto-release after X days with no objection per SRS; (2) blocked states: dispute
open,
chargeback risk, compliance hold; (3) idempotency: a crash mid-release must never
double-release (use a transaction + unique job key); (4) a reconciliation job
that
verifies ledger balance == sum of escrow holds daily and alerts on drift. Write
unit
tests for the state machine and a test that simulates a crash between debit and
credit. Show test output. Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build the milestone finance screens: web + mobile milestone list per project with
status (funded, work submitted, accepted, dispute window, released), a release
countdown,
and an escrow summary card (total held, releasable, blocked). Read-only screens
```

```
using
the typed client. Write tests for the countdown logic. Show output. Commit.
```

F1.4 Finance UI + Super Admin configuration UI

Lead: OpenCode (screens) with Claude review.

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build the two missing consoles at 65%: (1) Finance console: ledger journal viewer
(filterable by org, date, entry type), escrow overview, payouts queue with
approve/reject
and reason, invoices (reuse F1.2 screens), refund queue. (2) Super Admin
configuration:
feature flags editor, commission rates per category, dispute window length,
tender rules,
gateway credentials vault (write-only fields), environment switcher for
sandbox/live
with a confirmation modal. Every action logs to the admin audit table. Use the
design
system, RTL-safe. Write component tests for each table's filters and one action
flow
per screen. Show test output. Commit.
```

Claude Code prompt (review only):

Code

```
[Context Header]
Review the last merged PR (F1.4 consoles). Check only: permission enforcement
(can a
non-super-admin reach these routes/APIs?), audit logging completeness, and any
client
-side validation that must be duplicated server-side. List findings as
docs/reviews/f14-review.md with severity. Do not fix; I will route fixes.
```

F1.5 CI/CD: Dockerfiles, k8s manifests, pipelines (10% → 100%)

Lead: Claude Code (pipeline design) + OpenCode (per-service Dockerfiles).

Claude Code prompt:

Code

```
[Context Header]
Design and implement CI/CD: (1) multi-stage Dockerfiles for core API, nlp-
service,
vision-service, bim-service, web (standalone Next output), and a worker image;
(2)
docker-compose.dev.yml with postgres+postgis, opensearch, redis, minio; (3)
GitHub
Actions deploy pipeline: build, test, push to registry, deploy k8s manifests with
```

```
migrations job running before rollout; (4) k8s manifests for staging:
deployments,
H P A s, secrets via env refs, health probes matching each service's /health; (5)
a
rollback path (previous image tag pinned in deployment history). Document the
pipeline
in docs/ops/ci-cd.md. Show a successful local build of each image and the compose
stack coming up healthy. Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Write docs/ops/runbook.md from the existing CI/CD setup: how to deploy staging,
how to
read the pipeline, how to roll back, where logs live, how to run migrations
manually,
and the on-call checklist for a failed deploy. Plain language, screenshots
optional.
Also add .dockerignore files for every service. Commit.
```

F1.6 E2E happy paths (0 → the 6 journeys that matter)

Lead: OpenCode writes, Antigravity verifies in browser.

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Create Playwright E2E tests (web) covering the 6 core journeys end to end against
the
compose stack: (1) register org → invite user → login; (2) publish catalogue
product →
search finds it → add to cart → checkout (sandbox gateway mode); (3) create
tender →
bid → award → milestone → escrow funded; (4) order workforce crew → assign →
complete;
(5) book equipment/logistics trip → driver status updates; (6) invoice generated
→
ZATCA reported (sandbox) → visible in Finance console. Seed data per journey in
e2e/seed/. Each test asserts visible UI outcomes, not just HTTP 200. Run the
suite,
show passing output. Commit.
```

Antigravity prompt (after tests pass):

Code

```
[Context Header]
Run the app locally. Open it in your browser and manually click through journey 2
and
journey 3 (catalogue purchase and tender lifecycle). Screenshot each step. Report
any
place where the flow visually breaks, shows an error toast, or requires a page
reload
```

```
that a user wouldn't expect. Compare against the screens PDF flow for that
feature.
Output findings to docs/reviews/e2e-browser-report.md.
```

F1.7 PDPL compliance + in-region hosting + data purge

Lead: Claude Code.

Claude Code prompt:

Code

```
[Context Header]
Implement the PDPL obligations that are code (hosting itself is an ops task):
(1) data-subject endpoints: export all my data (JSON), delete my account with 30-
day
grace and cascade rules per SRS; (2) consent ledger: every consent event
(marketing,
location, biometric-equivalent for OCR documents) stored with timestamp, version,
source; consent checks enforced in the features that use them; (3) the 90-day
location
purge job (BullMQ repeatable) deleting trip location history older than 90 days,
verified by a test; (4) a data-retention table in code (one config) that drives
all
purge jobs; (5) document docs/privacy/pdpl-matrix.md mapping each PDPL article to
the
control implementing it. Write tests for export, delete-with-grace, and the purge
job.
Show output. Commit.
```

F1.8 Pen-test preparation and security hardening

Lead: Claude Code.

Claude Code prompt:

Code

```
[Context Header]
Prepare the platform for an external pen test: (1) rate limiting (Redis-backed)
on
auth, search, upload, payment and AI endpoints per SRS NFRs; (2) account lockout
+
breached-password check on auth; (3) security headers (CSP, HSTS, X-Frame-
Options) on
web and API; (4) file upload hardening: content-type sniffing, size limits,
virus-scan
hook, signed URLs with short TTL; (5) RBAC audit: write a script that walks every
endpoint and asserts a permission guard exists, output any endpoint missing one
to
docs/security/rbac-audit.md; (6) secrets scan pre-commit hook. Fix what the audit
finds. Show the audit going from X missing guards to 0. Commit.
```

LEVEL 2 — MARKETPLACE COMPLETION

The revenue core. Order within the level: F2.1 → F2.2 → F2.7 → F2.5 → F2.3 → F2.4 → F2.6 → F2.8. Bulk goes to OpenCode; anything touching money or algorithms goes to Claude.

F2.1 Catalogue completion (60% → 100%)

Lead: OpenCode + Nemotron Ultra 3.

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Feature: finish the Catalogue context. Read SRS Catalogue section + the screens
PDF
pages for catalogue/product. Build what is missing against the current 60%:
product
variants, unit-of-measure handling for construction materials, bulk pricing
tiers,
supplier pages with verified badges, category tree management UI, moderation
queue for
new listings. Reuse the design system. Every screen needs a Playwright component
test.
Run type-check + tests, show output. Commit. If a required contract
(types/endpoints)
does not exist, create a ticket note in docs/plan/contract-gaps.md and stop, do
not
invent endpoints.
```

Antigravity prompt:

Code

```
[Context Header]
Verify the Catalogue feature against the SRS and the screens PDF. Open the
running app,
walk every catalogue screen, screenshot, and produce docs/reviews/f21-report.md:
each SRS requirement marked done / partial / missing, with the screenshot as
evidence.
Pay special attention to RTL rendering and empty states.
```

F2.2 Search & discovery: geo-boost, filters, autocomplete polish

Lead: OpenCode bulk + Claude Code for the ranking logic.

Claude Code prompt:

Code

```
[Context Header]
Implement real search ranking in OpenSearch: (1) geo-distance boost from the
requester's location (PostGIS point → OpenSearch geo scoring), decay function per
SRS;
```

```
(2) combined score: relevance, rating, verification badge, distance, recency with
weights in configuration (not hardcoded); (3) typo tolerance and Arabic
analyzer/normalizer configuration for the product + supplier indices; (4) logging
of
query → click events into the analytics store for later ranking tuning. Write
integration tests with a seeded index proving: nearest verified supplier outranks
a
distant one for equal text match, Arabic query with typos still matches. Show
test
output. Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build search UI: results page with filter rail (category, distance slider,
rating,
verification, price range), sort control, map toggle showing clustered results
(PostGIS-backed endpoint), and an autocomplete component with debouncing, recent
searches and Arabic keyboard support. Mobile equivalents in Expo. Component tests
for
filters and autocomplete. Show test output. Commit.
```

F2.3 Workforce module (55% → 100%)

Lead: OpenCode (screens/CRUD) + Claude (assignment logic).

Claude Code prompt:

Code

```
[Context Header]
Implement the workforce assignment engine: given a job (trade, location, date,
crew
size), rank available workers by skill match, distance, rating and past no-show
rate;
support manual override and bulk assign; enforce max-hours/day and rest rules;
emit
BullMQ reminders at T-24h and T-2h. Replace any findFirst-style shortcut with
this
engine. Tests: ranking correctness, no double-booking, override logging. Show
output.
Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build workforce screens: worker profile + documents + availability calendar,
company
job board (post/assign/complete), worker-side job list with accept/reject and
check-in
via geolocation, timesheet approval flow, ratings. Web + mobile. Component tests
```

```
per
screen. Show output. Commit.
```

F2.4 Logistics & equipment: real dispatch, maps, geofences (55% → 100%)

Lead: Claude Code (dispatch + maps integration). This is where the report's fake dispatch and missing map provider get fixed for good.

Claude Code prompt:

Code

```
[Context Header]
Fix logistics properly: (1) choose and integrate a map/routing provider that
works in
Saudi Arabia (e.g. Google Maps Platform or Mapbox; abstract behind a MapsProvider
interface so the provider is swappable); implement geocoding, routing, ETA and
distance matrix; (2) replace findFirst dispatch with a real algorithm: candidate
drivers = online + capacity + equipment fit, ranked by pickup ETA + rating;
dispatch
offer goes to top-N with timeout cascade to next driver; (3) geofences: pickup/
dropoff zones checked on the worker, with arrival/departure events; (4) mobile
location tracking: the currently-404 PATCH /api/trips/[id]/location and
[id]/location endpoints must exist, accept batched points, and the 90-day purge
from
F1.7 covers them; (5) wire the live map on customer and driver screens to the
WebSocket
gateway. Integration tests for dispatch cascade and geofence events. Show output.
Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build logistics screens on top of the new dispatch engine: dispatch console (live
map,
pending offers, driver list, manual reassign), trip detail with timeline +
location
trail, equipment rental calendar with availability, driver mobile screen (current
trip,
navigation hand-off button, status buttons, proof-of-delivery photo upload).
Component
tests. Show output. Commit.
```

F2.5 Tendering: real sealed bids + award flow (75% → 100%)

Lead: Claude Code (crypto was already fixed in F0.2; now finish the flow).

Claude Code prompt:

Code

```
[Context Header]
Complete tendering: (1) full lifecycle states (draft, published, Q&A, closed,
evaluated,
awarded, cancelled) with the transitions enforced server-side; (2) sealed-bid
reveal
 ceremony: after close, a logged decryption event per bid with hash pre-
commitment
published at bid time (bid hash visible to all before close proves non-
tampering);
(3) evaluation matrix (weighted criteria) with per-evaluator scoring and conflict
lock (evaluator cannot see others' scores before submitting); (4) award + auto-
create
contract/project + initial milestones, tied to the F1.3 escrow. Tests for the
reveal
ceremony and the conflict lock. Show output. Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build tender screens: buyer side (create tender wizard with BOQ upload,
evaluation
matrix builder, bid comparison table, award confirmation), supplier side (browse,
filter, bid with document upload, my bids with statuses), public tender page per
the
screens PDF. Web + mobile. Component tests. Show output. Commit.
```

F2.6 Disputes & resolution (55% → 100%)

Lead: OpenCode screens + Claude state machine review.

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build the disputes flow: open dispute from a milestone/order (reason categories,
evidence upload, amount in question), evidence exchange thread, admin resolution
console (assign mediator, timeline, decision: refund/ partial refund/ release/
split),
and notifications at each state change to both parties. Follow the escrow states
from
F1.3 exactly, dispute open blocks release. Component tests for the thread and the
decision flow. Show output. Commit.
```

Claude Code prompt:

Code

```
[Context Header]
Review the disputes PR for money-safety: dispute open MUST block escrow release,
a decision MUST produce exactly one ledger adjustment (idempotent), and partial
refund
math must balance the journal. Write the three tests that prove each. Fix
```

```
findings.
Show output. Commit.
```

F2.7 Organizations & permissions completion (70% → 100%)

Lead: OpenCode.

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Finish Identity & Organisations: org settings (profile, branding, trade license
upload
with expiry reminders), member management (invite by email/phone, roles,
deactivate),
team hierarchy for large orgs, org switcher for multi-org users, API keys per org
with
scopes and rotation. Enforce the permission matrix from the SRS in UI and rely on
the
server guards from F1.8. Component tests. Show output. Commit.
```

F2.8 Mobile parity (65% → 100%)

Lead: OpenCode, verified by Antigravity.

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Close the mobile gaps: every feature built in Level 2 gets its mobile counterpart
in
Expo with Expo Router, sharing the same contracts package. Priorities: tender
browsing
and bidding, workforce jobs, logistics driver app screens, dispute thread,
finance/
milestone status, push notifications wiring (expo-notifications: permissions,
token
registration, deep-link handling per notification type). Match the screens PDF
mobile
pages. Verify each screen against its web counterpart's data contract. Commit per
feature slice.
```

Antigravity prompt:

Code

```
[Context Header]
Mobile parity audit: for each Level 2 feature, compare web vs mobile coverage
against
the SRS and screens PDF. Output docs/reviews/f28-parity.md: feature, web status,
mobile status, gaps. Rank gaps by user impact.
```

LEVEL 3 — AI MODULES

Build order: F3.5 first (the evaluation harness), then F3.3, F3.2, F3.1, F3.4. You cannot improve what you cannot measure, and the report shows image "detection" is currently OCR-word-boxes pretending to be detection.

F3.5 Vision/NLP evaluation harness (do this first)

Lead: Claude Code.

Claude Code prompt:

Code

```
[Context Header]
Build an evaluation harness for the AI services (Python, pytest): (1) a golden
dataset
of 100+ samples per task: material photos with ground-truth labels, document
scans
including handwritten Arabic with ground-truth text, BIM files with known
quantities;
store under ai/evals/data/; (2) a runner that scores each service against the set
(accuracy, precision/recall for detection; CER/WER for OCR; quantity error % for
BIM);
(3) CI job running a 20-sample smoke set on every AI-service PR, full set
nightly;
(4) an eval report artifact (markdown) per run. This harness is the acceptance
gate
for F3.1-F3.4. Show one full report generated from the current services. Commit.
```

F3.3 OCR & handwriting service hardening (70% → real)

Lead: Claude Code.

Claude Code prompt:

Code

```
[Context Header]
Harden vision-service document processing: (1) replace the OCR-word-box
approximation
with a real pipeline: page detection → text region detection → recognition
(choose a
maintained engine that handles printed Arabic + English; add a handwriting
recognition
model for delivery notes and daily reports); (2) structured extraction: map
recognized
text to the SRS document schemas (invoice fields, delivery note lines,
quantities,
signatures); (3) confidence per field with a low-confidence human-review queue
(BullMQ + admin screen ticket); (4) image auto-deletion after processing per
PDPL.
Measure with the F3.5 harness before and after: report CER/WER improvement. Show
the
eval reports. Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build the document-review UI: queue of low-confidence extractions, side-by-side
original image and extracted fields, inline correction, approve/redo, review
stats
dashboard (throughput, correction rate by field). Web only. Component tests. Show
output. Commit.
```

F3.2 Real material image detection (65% → real detection)

Lead: Claude Code.

Claude Code prompt:

Code

```
[Context Header]
Replace OCR-word-boxes-as-detection with real material detection in vision-
service:
(1) select and integrate an open-weights detection model (e.g. a YOLO-family or
DETR-family model) fine-tunable on construction material classes from the SRS
catalogue
(cement, steel rebar, blocks, tiles, pipes, insulation, etc.); (2) run ON-device-
free
server inference with batching, GPU/CPU config via env; (3) map detections to
catalogue
products via embedding similarity (pgvector) with a confidence threshold and
low-confidence fallback to search suggestions instead of wrong labels; (4) add
the
training/fine-tuning pipeline skeleton (dataset format, training script, export)
so
the client's own photos can improve it over time. Gate acceptance on the F3.5
harness:
detection mAP must beat the current word-box heuristic on the golden set. Show
the
comparison report. Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build the photo-to-product UX: upload/camera capture with crop, detection overlay
boxes with labels + confidence, one-tap "add to quote" per detection, corrections
feeding back as mislabeled-data reports. Web + mobile camera screen (Expo camera,
already configured but unused). Component tests with mocked detections. Show
output.
Commit.
```

F3.1 AI assistant (10% → real, RAG + function calling)

Lead: Claude Code. This is the flagship AI feature (FR-AI-01..05).

Claude Code prompt:

Code

```
[Context Header]
Build the AI assistant properly in nlp-service + core API: (1) RAG: index
catalogue
products, suppliers, tender documents and SRS-derived help content into pgvector
+
OpenSearch hybrid retrieval (Arabic + English embedding pipeline, chunking per
doc
type); (2) function calling so the assistant can ACT: search products, create
cart/
quote, check order status, draft tender, fetch invoice, escalate to human, with a
permission pass on every tool call (the assistant can never exceed the user's
RBAC);
(3) scope guards per role (buyer/contractor/supplier/admin see different tools);
(4) full audit log of every tool call with parameters and result; (5) streaming
responses over WebSocket; (6) an eval suite (F3.5-style) of 50 golden questions
with
expected tool calls and answers, run in CI. Show the eval report passing. Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build the assistant chat UI: web widget + mobile screen; message list with
streaming
tokens, tool-call cards ("Searching products...", results preview), suggested
prompts per page context, voice input button (use existing voice flow), handoff
to
human support, RTL-safe rendering, markdown with product cards inline. Component
tests with a mocked stream. Show output. Commit.
```

F3.4 BIM service: IfcOpenShell quantity extraction

Lead: Claude Code.

Claude Code prompt:

Code

```
[Context Header]
Harden bim-service to production: (1) IFC upload → parse with IfcOpenShell →
extract
quantities per element type (concrete volume, steel tonnage, block counts, areas)
mapped to the catalogue's units of measure; (2) BOQ generation: element
quantities →
draft bill of quantities with unit prices from the catalogue; (3) validation:
missing
property sets, unsupported IFC schema versions, huge-file streaming handling;
(4) expose results via the typed API contract (new endpoints if needed,
```

```
regenerate
contracts); (5) eval vs the F3.5 golden BIM set: quantity error under the SRS
threshold. Show the eval report. Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build the BIM UI: upload with progress, 3D preview (web, lightweight viewer or
rendered snapshots), quantity table by category with drill-down, "generate BOQ"
action creating a draft in the tendering module, extraction errors panel.
Component
tests. Show output. Commit.
```

LEVEL 4 — POLISH & SCALE

F4.1 i18n / RTL audit and completion (55% → 100%)

Lead: OpenCode. Massive, mechanical, perfect for Nemotron Ultra 3 in parallel slices.

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Complete internationalization: (1) extract every hardcoded string in web + mobile
into
ar.json and en.json message files per feature folder (run per feature slice, one
slice
per session: auth, catalogue, search, payments, tenders, workforce, logistics,
disputes,
finance, admin); (2) wrap the app in the i18n provider with locale switch,
RTL/LTR
direction handling, Arabic-Indic numeral option per SRS; (3) an audit script that
fails CI if a string literal appears in a component; (4) fix every broken RTL
layout
found (icons flipping, padding sides, chart directions). Run the audit script to
zero.
Commit per slice.
```

F4.2 Accessibility (25% → WCAG AA)

Lead: OpenCode + Antigravity audit.

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Bring web + mobile to WCAG 2.1 AA for the core journeys: semantic landmarks,
focus
management in modals and wizards, keyboard-only checkout and tender flows,
visible
```

```
focus styles, color contrast fixes against the design tokens, alt text on all
images,
ARIA on custom components, mobile screen-reader labels (Expo AccessibilityInfo).
Add
axe-core to the Playwright suite, fail CI on critical violations. Run and show
the
report going green. Commit per slice.
```

Antigravity prompt:

Code

```
[Context Header]
Accessibility walkthrough: navigate the whole purchase journey using keyboard
only,
screenshot the focus path, and try the same with high-contrast and 200% zoom.
Report
blockers to docs/reviews/f42-a11y.md ranked by severity.
```

F4.3 Analytics & reporting (15% → real)

Lead: OpenCode (dashboards) + Claude (event pipeline).

Claude Code prompt:

Code

```
[Context Header]
Implement the analytics event pipeline: (1) define the event schema per SRS
(search,
view, add-to-cart, bid, order, milestone, trip, invoice events) with a versioned
contract; (2) emit from core API services via the outbox pattern to a queue, then
into
an analytics store; (3) daily rollup jobs (GMV, active orgs, conversion per
funnel,
dispatch times, dispute rate); (4) no PII in analytics: enforce a scrubbing pass
with
a test. Show the pipeline flowing one event end to end in the compose stack.
Commit.
```

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build dashboards: admin overview (GMV, orders, tenders, active users with trend
charts), supplier sales dashboard, funnel views per F4.3 events, export to CSV,
date-range and org filters, RTL-safe charts. Use the rollup endpoints. Component
tests for filters. Show output. Commit.
```

F4.4 Content operations (15% → real)

Lead: OpenCode.

OpenCode + Nemotron Ultra 3 prompt:

Code

```
[Context Header]
Build content ops: CMS for banners, category imagery, help articles (Arabic +
English),
legal pages with versioning; scheduled publish/unpublish; slug-based public pages
rendered server-side with SEO meta + Arabic hreflang; review/approval workflow
(draft → review → published). Component tests for the workflow. Show output.
Commit.
```

F4.5 Performance & load testing against the NFRs

Lead: Claude Code.

Claude Code prompt:

Code

```
[Context Header]
Prove the SRS NFRs: (1) load-test scripts (k6 or Playwright-trace-based) for the
top endpoints: search P95 under target at 10k concurrent sessions per NFR-SCL-01,
checkout path, tender close burst, WebSocket fan-out; (2) run against the compose
stack + staging, capture results; (3) fix the top 5 bottlenecks found (likely:
missing
indices, N+1 queries, unpaginated lists, cold AI-service starts, missing
caching);
re-run and show before/after; (4) add the nightly load-test job with regression
alerts. Show before/after numbers. Commit.
```

F4.6 Real-time reliability (WebSocket gateway)

Lead: Claude Code.

Claude Code prompt:

Code

```
[Context Header]
Harden real-time: (1) reconnection with exponential backoff + resumable event
replay
from a per-user event cursor so nothing is lost during a disconnect; (2) presence
tracking for dispatch and chat; (3) subscription authorization per room
(org/project/
trip/tender) enforced server-side with tests proving a cross-tenant join fails;
(4) horizontal scale path (Redis adapter) verified with two instances in compose.
Show tests passing including the cross-tenant denial. Commit.
```

The daily loop, applied to this plan

> **Morning (Antigravity):** take the current level's next feature, run its Antigravity prompt, get tickets + acceptance tests.
>
> **Split:** money/security/algorithm tickets → Claude Code. Screens/CRUD/tests/i18n → OpenCode + Nemotron Ultra 3, in parallel slices.

Build: one ticket per session, Context Header always, proof always, commit always.

Gate: CI must be green before the next ticket. Contract changes regenerate the client, never hand-edited.

Cross-review: Claude reviews what Nemotron built (F1.4 style), Nemotron's harness measures what Claude built (F3.5 style).

Verify: Antigravity opens the running app and clicks the feature. Screenshot or it didn't happen.

Friday: Antigravity re-audits the whole level against the SRS: done / partial / missing. That report decides next week's order.

Budget shape for this plan

Claude Code carries about a quarter of the work and nearly all of the risk: Max 5x ($100) is the right tier for Levels 1 and 3; Pro ($20) may hold for Level 2/4 weeks if you route ruthlessly. OpenCode with Nemotron Ultra 3 is free, which is why the bulk of Level 2 and 4 lands there. Antigravity rides your Google plan and costs nothing extra. Keep a small Z.ai GLM ($18) or DeepSeek balance as the OpenCode fallback for when Nemotron fails a ticket twice.

The three rules that decide success

No feature is done on a claim. Test output, eval report, or browser screenshot.

Money, crypto, auth, and AI logic stay on Claude Code. Bulk stays on Nemotron.

Antigravity audits weekly against the SRS. That audit is what would have caught the fake social adapters and the findFirst dispatch months earlier.